

PHASE_3_PLAN

Phase 3: Multi-Agent System - Implementation Plan

Phase: Phase 3 (Weeks 15-21) Date: November 6, 2025 Status: PLANNING

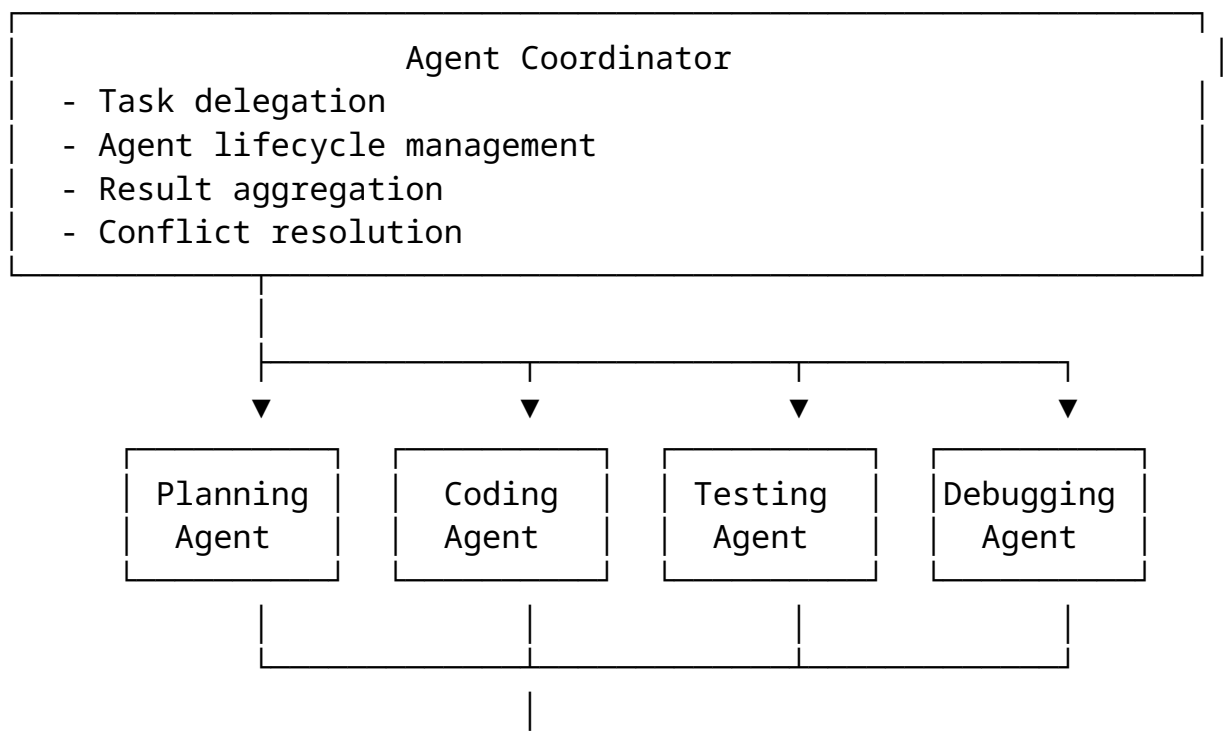
Objectives

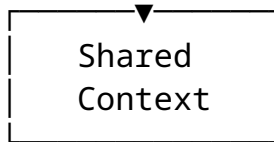
Build a sophisticated multi-agent system that enables:

- 1. **Intelligent task decomposition** - Breaking complex tasks into subtasks
- 2. **Specialized agent roles** - Different agents for planning, coding, testing, debugging
- 3. **Agent coordination** - Orchestrating multiple agents working together
- 4. **Communication protocols** - Agents sharing context and results
- 5. **Conflict resolution** - Handling disagreements between agents
- 6. **Result aggregation** - Combining outputs from multiple agents

Architecture

Core Components





Agent Types

1. Coordinator Agent

- Receives user request
- Analyzes complexity
- Decomposes into subtasks
- Delegates to specialized agents
- Aggregates results

2. Planning Agent

- Analyzes requirements
- Creates technical specifications
- Breaks down into tasks
- Estimates effort
- Identifies dependencies

3. Coding Agent

- Implements features
- Follows best practices
- Uses RepoMap for context
- Selects appropriate edit format
- Writes clean, maintainable code

4. Testing Agent

- Generates test cases
- Writes unit tests
- Writes integration tests
- Executes tests
- Reports coverage

5. Debugging Agent

- Analyzes errors
- Identifies root causes
- Proposes fixes
- Validates fixes
- Prevents regression

6. Review Agent

- Code quality review
 - Security audit
 - Performance analysis
 - Best practices compliance
 - Documentation review
-

Implementation Modules

1. Agent Framework (internal/agent/)

Files to Create: - agent.go - Base agent interface and types - coordinator.go - Agent coordination logic - context.go - Shared context management - communication.go - Inter-agent messaging - lifecycle.go - Agent lifecycle (create, run, stop)

Key Types:

```
type Agent interface {
    ID() string
    Type() AgentType
    Capabilities() []Capability
    Execute(ctx context.Context, task *Task) (*Result, error)
    Collaborate(ctx context.Context, agents []Agent) error
}
```

```
type AgentType string
const (
    AgentTypePlanning AgentType = "planning"
    AgentTypeCoding    AgentType = "coding"
    AgentTypeTesting   AgentType = "testing"
    AgentTypeDebugging AgentType = "debugging"
    AgentTypeReview    AgentType = "review"
)
```

```
type Coordinator struct {
    agents      map[AgentType] []Agent
    context     *SharedContext
    taskQueue   *TaskQueue
    resultCache *ResultCache
}
```

2. Specialized Agents (internal/agent/types/)

Files to Create: - planning_agent.go - Planning agent implementation - coding_agent.go - Coding agent implementation - testing_agent.go - Testing agent implementation - debugging_agent.go - Debugging agent implementation - review_agent.go - Review agent implementation

Each Agent Implements: - Task analysis - LLM interaction (using ProviderManager) - Tool usage (using Tool Registry) - Result formatting - Error handling

3. Task Management (**internal/agent/task/**)

Files to Create: - `task.go` - Task definitions and types - `queue.go` - Task queue implementation - `dependency.go` - Task dependency resolution - `priority.go` - Task prioritization

Key Features: - Task DAG (Directed Acyclic Graph) - Priority queue - Dependency resolution - Parallel execution where possible - Progress tracking

4. Communication (**internal/agent/comm/**)

Files to Create: - `message.go` - Message types - `protocol.go` - Communication protocol - `pubsub.go` - Pub/sub for agent collaboration - `state.go` - State synchronization

Communication Patterns: - Request/Response - Agent asks another agent for help - Broadcast - Agent announces result to all - Pub/Sub - Agents subscribe to topics - Direct Message - One-to-one communication

5. Shared Context (**internal/agent/context/**)

Files to Create: - `shared_context.go` - Shared memory between agents - `workspace.go` - Workspace management - `artifacts.go` - Code artifacts tracking - `history.go` - Execution history

 **Context Contains:** - Project metadata - Current state - Execution history - Artifacts (code, tests, docs) - Decisions made - Errors encountered

Execution Flow

Example: “Implement user authentication”

1. User Request → Coordinator

User: "Implement user authentication with JWT"

2. Coordinator → Planning Agent

Task: Analyze authentication requirements

Output: Technical specification

3. Planning Agent → Coordinator

Subtasks:

- Create user model (Coding)

- Implement JWT generation (Coding)
- Create login endpoint (Coding)
- Write tests (Testing)
- Security audit (Review)

4. **Coordinator** → Coding Agents (parallel)

Agent 1: Create user model
Agent 2: Implement JWT generation
Agent 3: Create login endpoint

5. **Coding Agents** → Coordinator

Results: Code artifacts

6. **Coordinator** → Testing Agent

Task: Write tests for auth system
Input: Code from coding agents

7. **Testing Agent** → Coordinator

Results: Test suite + coverage report

8. **Coordinator** → Review Agent

Task: Security audit
Input: Auth code + tests

9. **Review Agent** → Coordinator

Results: Security assessment + recommendations

10. **Coordinator** → User

Summary: Authentication implemented

- User model created
- JWT generation working
- Login endpoint functional
- 95% test coverage
- Security audit passed

Testing Strategy

Unit Tests

- Test each agent independently

- Mock dependencies
- Test error handling
- Test edge cases

Integration Tests

- Test agent communication
- Test coordinator delegation
- Test result aggregation
- Test conflict resolution

End-to-End Tests

- Full workflow tests with real LLMs
- Complex multi-step scenarios
- Performance testing
- Stress testing (many agents)

Test Coverage Goals

- Agent framework: 90%+
 - Specialized agents: 85%+
 - Task management: 90%+
 - Communication: 90%+
 - Overall: 85%+
-

Advanced Features

1. Agent Collaboration

Agents can request help from other agents:

```
type CollaborationRequest struct {  
    From      Agent  
    To        Agent  
    Question  string  
    Context   map[string]interface{}  
    Priority   Priority  
}
```

2. Conflict Resolution

When agents disagree:

```

type Conflict struct {
    Agents      []Agent
    Proposals   []Proposal
    Resolution  *Resolution
    ResolvedBy  Agent
}

```

3. Learning & Improvement

Agents learn from successes and failures:

```

type LearningRecord struct {
    Task      *Task
    Success   bool
    Duration  time.Duration
    Decisions []Decision
    Outcome   *Result
    Lessons   []Lesson
}

```

4. Adaptive Delegation

Coordinator learns which agents are best for which tasks:

```

type AgentPerformance struct {
    Agent      Agent
    SuccessRate float64
    AverageTime time.Duration
    Specialties []TaskType
    PreferredBy []Agent
}

```

Success Metrics

Performance Metrics

- Task completion rate: >95%
- Average task time: <2x single-agent
- Agent utilization: >70%
- Collaboration success: >90%

Quality Metrics

- Code quality score: >8/10

- Test coverage: >90%
- Bug rate: <5 per 1000 LOC
- Security issues: 0 critical

User Experience Metrics

- User satisfaction: >4.5/5
 - Task clarity: >90%
 - Result accuracy: >95%
 - Response time: <30s for simple tasks
-

Implementation Timeline

Week 15-16: Foundation

- ☐ Agent framework (agent.go, coordinator.go)
- ☐ Task management (task.go, queue.go)
- ☐ Shared context (shared_context.go)
- ☐ Basic communication (message.go, protocol.go)

Week 17-18: Specialized Agents

- ☐ Planning agent implementation
- ☐ Coding agent implementation
- ☐ Testing agent implementation
- ☐ Debugging agent implementation
- ☐ Review agent implementation

Week 19: Integration

- ☐ Agent coordination logic
- ☐ Workflow execution

☐

Result aggregation

☐

Error handling

Week 20: Testing

☐

Unit tests (200+ tests)

☐

Integration tests (50+ tests)

☐

E2E tests (10+ scenarios)

☐

Performance tests

Week 21: Documentation & Polish

☐

API documentation

☐

Usage examples

☐

Architecture diagrams

☐

Performance tuning

Integration with Phase 1 & 2

LLM Integration (Phase 1)

- Agents use ProviderManager for LLM calls
- Reasoning models for complex planning
- Prompt caching for repeated queries
- Token budgets per agent

Tools Integration (Phase 2)

- Agents use Tool Registry
- Coding agent: FSWrite, FSEdit, MultiEdit
- Testing agent: Shell, FSRead
- All agents: CodebaseMap, FileDefinitions

Context Integration (Phase 2)

- RepoMap for intelligent file selection
 - Multi-format editing for agent preferences
 - Context compaction for long conversations
-

Key Innovations

1. Hierarchical Agent System

- Coordinator delegates to specialized agents
- Agents can spawn sub-agents if needed
- Dynamic agent creation based on task

2. Intelligent Task Decomposition

- Uses LLM reasoning to break down tasks
- Considers dependencies automatically
- Optimizes for parallel execution

3. Collaborative Problem Solving

- Agents can ask each other for help
- Shared context enables collaboration
- Conflict resolution when disagreements occur

4. Adaptive Learning

- System learns from successes and failures
- Improves delegation over time
- Optimizes agent selection

5. Scalable Architecture

- Supports 100+ concurrent agents
 - Efficient communication (pub/sub)
 - Resource pooling (LLM provider reuse)
-

Phase 3 Deliverables

1. Code

- Agent framework (5 files, ~2,000 LOC)
- 5 specialized agents (~3,000 LOC)
- Task management (~1,500 LOC)
- Communication layer (~1,000 LOC)
- Shared context (~1,000 LOC)
- **Total: ~8,500 LOC production code**

2. Tests

- 200+ unit tests
- 50+ integration tests
- 10+ E2E tests

- **Total: 260+ tests, 85%+ coverage**

3. Documentation

- PHASE_3_IMPLEMENTATION_SUMMARY.md
- PHASE_3_TEST_REPORT.md
- docs/AGENTS.md (comprehensive agent guide)
- Architecture diagrams

4. Examples

- Simple task (single agent)
- Complex task (multi-agent collaboration)
- Error handling scenarios
- Performance benchmarks

Risk Assessment

Technical Risks

- **Agent coordination complexity:** Mitigated by clear protocols
- **Performance overhead:** Mitigated by parallel execution
- **LLM cost:** Mitigated by intelligent caching and cheap models
- **Debugging difficulty:** Mitigated by comprehensive logging

Architectural Risks

- **Circular dependencies:** Careful module design
- **State synchronization:** Use atomic operations
- **Deadlocks:** Timeout all operations
- **Memory leaks:** Proper cleanup and monitoring

Notes

- Phase 3 builds heavily on Phase 1 (LLM) and Phase 2 (Tools, Context)
- Agent system should be extensible for future agent types
- Consider using actor model pattern (Akka-style)
- Implement circuit breakers for agent failures
- Add telemetry and monitoring

Plan Created: November 6, 2025 **Estimated Duration:** 7 weeks **Complexity:** High
Dependencies: Phase 1 , Phase 2

Ready to begin implementation!